

Лабораторная работа №13

Кац Григорий Михайлович

Содержание

1. Цель работы	5
2. Задание	6
3. Теоретическое введение	7
4. Выполнение лабораторной работы	8
5. Выводы	13
Список литературы	14

Список иллюстраций

1	Первый скрипт	9
2	Второй скрипт	10
3	Третий скрипт	11
4	Четвертый скрипт	12
5	Запуск	12

Список таблиц

1. Цель работы

Освоить базовые принципы программирования в оболочке UNIX и научиться писать сложные сценарии (командные файлы), используя логические конструкции и циклы.

2. Задание

1. Используя команды `getopts` `grep`, написать командный файл, который анализирует командную строку с ключами: `-iinputfile` — прочитать данные из указанного файла; `-ooutputfile` — вывести данные в указанный файл; `-r` шаблон — указать шаблон для поиска; `-C` — различать большие и малые буквы; `-n` — выдавать номера строк. а затем ищет в указанном файле нужные строки, определяемые ключом `-r`.
2. Написать на языке Си программу, которая вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю. Затем программа завершается с помощью функции `exit(n)`, передавая информацию в о коде завершения в оболочку. Командный файл должен вызывать эту программу и, проанализировав с помощью команды `$?`, выдать сообщение о том, какое число было введено.
3. Написать командный файл, создающий указанное число файлов, пронумерованных последовательно от 1 до $\square$ (например `1.tmp`, `2.tmp`, `3.tmp`, `4.tmp` и т.д.). Число файлов, которые необходимо создать, передаётся в аргументы командной строки. Этот же командный файл должен уметь удалять все созданные им файлы (если они существуют).
4. Написать командный файл, который с помощью команды `tar` запаковывает в архив все файлы в указанной директории. Модифицировать его так, чтобы запаковывались только те файлы, которые были изменены менее недели тому назад (использовать команду `find`).

3. Теоретическое введение

Bash — это популярная командная оболочка UNIX, обеспечивающая интерактивную работу с командами и возможность написания скриптов для автоматизации. Её ключевые особенности: обработка простых и сложных команд (конвейеры, перенаправление, if/else, while), удобный ввод с автодополнением и историей, а также создание исполняемых скриптов с циклами и функциями. Работает в Unix/Linux и Windows (через WSL).

4. Выполнение лабораторной работы

1. Используя команды `getopts` `grep`, написать командный файл, который анализирует командную строку с ключами: `-i`inputfile — прочитать данные из указанного файла; `-o`outputfile — вывести данные в указанный файл; `-r`шаблон — указать шаблон для поиска; `-C` — различать большие и малые буквы; `-n` — выдавать номера строк. а затем ищет в указанном файле нужные строки, определяемые ключом `-r`.

```
#include <stdio.h>
#include <stdlib.h>

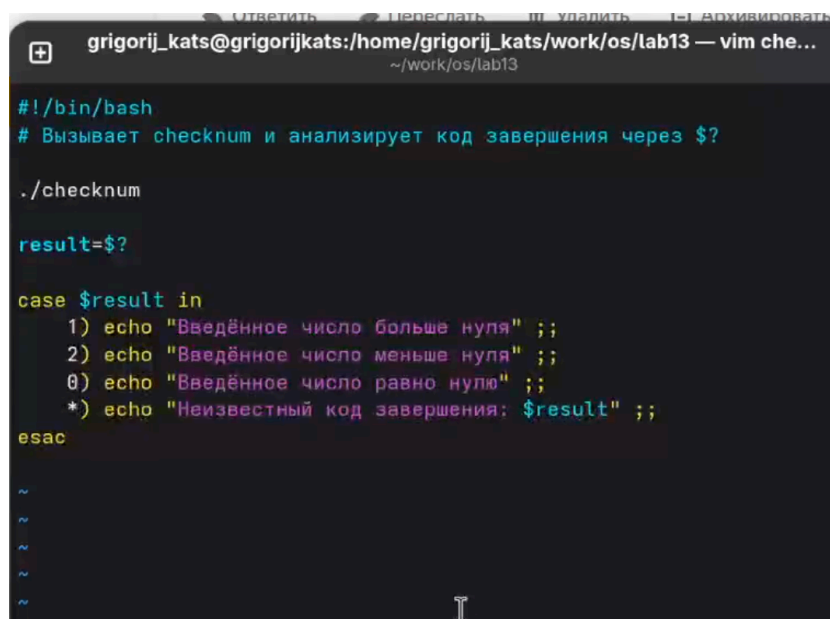
int main() {
    int num;
    printf("Введите число: ");
    scanf("%d", &num);

    if (num > 0) {
        exit(1);
    } else if (num < 0) {
        exit(2);
    } else {
        exit(0);
    }

    return 0;
}
```

Рис. 1: Первый скрипт

2. Написать на языке Си программу, которая вводит число и определяет, является ли оно больше нуля, меньше нуля или равно нулю. Затем программа завершается с помощью функции `exit(n)`, передавая информацию в о коде завершения в оболочку. Командный файл должен вызывать эту программу и, проанализировав с помощью команды `$?`, выдать сообщение о том, какое число было введено.



```
grigorij_kats@grigorijkats:/home/grigorij_kats/work/os/lab13 — vim che...
~/work/os/lab13

#!/bin/bash
# Вызывает checknum и анализирует код завершения через $?

./checknum

result=$?

case $result in
  1) echo "Введённое число больше нуля" ;;
  2) echo "Введённое число меньше нуля" ;;
  0) echo "Введённое число равно нулю" ;;
  *) echo "Неизвестный код завершения: $result" ;;
esac

~
~
~
~
~
```

Рис. 2: Второй скрипт

3. Написать командный файл, создающий указанное число файлов, пронумерованных последовательно от 1 до n (например 1.tmp, 2.tmp, 3.tmp, 4.tmp и т.д.). Число файлов, которые необходимо создать, передаётся в аргументы командной строки. Этот же командный файл должен уметь удалять все созданные им файлы (если они существуют).

```
grigorij_kats@grigorijkats:/home/grigorij_kats/work/os/lab13 — vim ma...
~/work/os/lab13

i=1
while [ $i -le $N ]
do
    touch "${i}.tmp"
    echo "Создан файл ${i}.tmp"
    i=$((i+1))
done
elif [ "$action" == "delete" ]; then
i=1
while [ $i -le $N ]
do
    if [ -f "${i}.tmp" ]; then
        rm "${i}.tmp"
        echo "Удалён файл ${i}.tmp"
    else
        echo "Файл ${i}.tmp не существует"
    fi
    i=$((i+1))
done
else
echo "Неизвестное действие: $action. Используйте create или delete"
exit 1
```

Рис. 3: Третий скрипт

4. Написать командный файл, который с помощью команды `tag` запаковывает в архив все файлы в указанной директории. Модифицировать его так, чтобы запаковывались только те файлы, которые были изменены менее недели тому назад (использовать команду `find`).

```
grigorij_kats@grigorijkats:/home/grigorij_kats/work/os/lab13 — vim arc...
~/work/os/lab13

#!/bin/bash
# Архивирует все файлы указанной директории через tar
# Использование: ./archive_dir.sh директория архив.tar.gz

DIR=$1
ARCHIVE=$2

if [ -z "$DIR" ] || [ -z "$ARCHIVE" ]; then
    echo "Использование: $0 директория архив.tar.gz"
    exit 1
fi

if [ ! -d "$DIR" ]; then
    echo "Ошибка: $DIR не является каталогом"
    exit 1
fi

tar -czvf "$ARCHIVE" -C "$DIR" .
echo "Архив создан: $ARCHIVE"

~
~
~
~
```

Рис. 4: Четвертый скрипт

Делаю файлы исполняемыми и запускаю.

```
grigorij_kats@grigorijkats:/home/grigorij_kats/work/os/lab13
~/work/os/lab13

chmod: missing operand after '+xarchive_recent.sh'
Try 'chmod --help' for more information.
grigorij_kats@grigorijkats:~/work/os/lab13$ chmod +x archive_recent.sh
grigorij_kats@grigorijkats:~/work/os/lab13$ mkdir -p testdir
grigorij_kats@grigorijkats:~/work/os/lab13$ touch testdir/new_file.txt
grigorij_kats@grigorijkats:~/work/os/lab13$ touch -d "10 days ago" testdir/old_file.txt
grigorij_kats@grigorijkats:~/work/os/lab13$ ./archive_dir.sh testdir full_archive.tar.gz
./
./new_file.txt
./old_file.txt
Архив создан: full_archive.tar.gz
grigorij_kats@grigorijkats:~/work/os/lab13$ ./archive_recent.sh testdir recent_archive.tar.gz
testdir/new_file.txt
Архив создан: recent_archive.tar.gz (только файлы младше недели)
grigorij_kats@grigorijkats:~/work/os/lab13$ tar -tvf full_archive.tar.gz
drwxr-xr-x grigorij_kats/grigorij_kats 0 2026-09-04 12:31 ./
-rw-r--r-- grigorij_kats/grigorij_kats 0 2026-09-04 12:30 ./new_file.txt
-rw-r--r-- grigorij_kats/grigorij_kats 0 2026-08-25 12:31 ./old_file.txt
grigorij_kats@grigorijkats:~/work/os/lab13$ tar -tvf recent_archive.tar.gz
-rw-r--r-- grigorij_kats/grigorij_kats 0 2026-09-04 12:30 testdir/new_file.txt
grigorij_kats@grigorijkats:~/work/os/lab13$
```

Рис. 5: Запуск

5. Выводы

Мы освоили базу программирования в оболочке UNIX и приобрели навыки написания усложнённых командных скриптов, применяя логические конструкции и циклы.

Список литературы